



# Informe Técnico: CAG Lab — Examen Final Módulo 3

**Proyecto:** Examen Final Módulo 3 — Asistente educativo con Context-Augmented Generation

**Autor:** César Estrada

**Fecha:** Junio 2026

**Stack:** Python 3.14+ (stdlib) · Frontend vanilla (HTML/CSS/JS) · unittest

---

## 1. Resumen Ejecutivo

Este proyecto implementa un asistente educativo que combina dos técnicas de aumentación de respuestas:

- **RAG (Retrieval-Augmented Generation):** Recupera fragmentos relevantes de una base de conocimiento del curso.
- **CAG (Context-Augmented Generation):** Retiene contexto persistente del usuario (preferencias, audiencia, idioma) y lo inyecta en las respuestas.

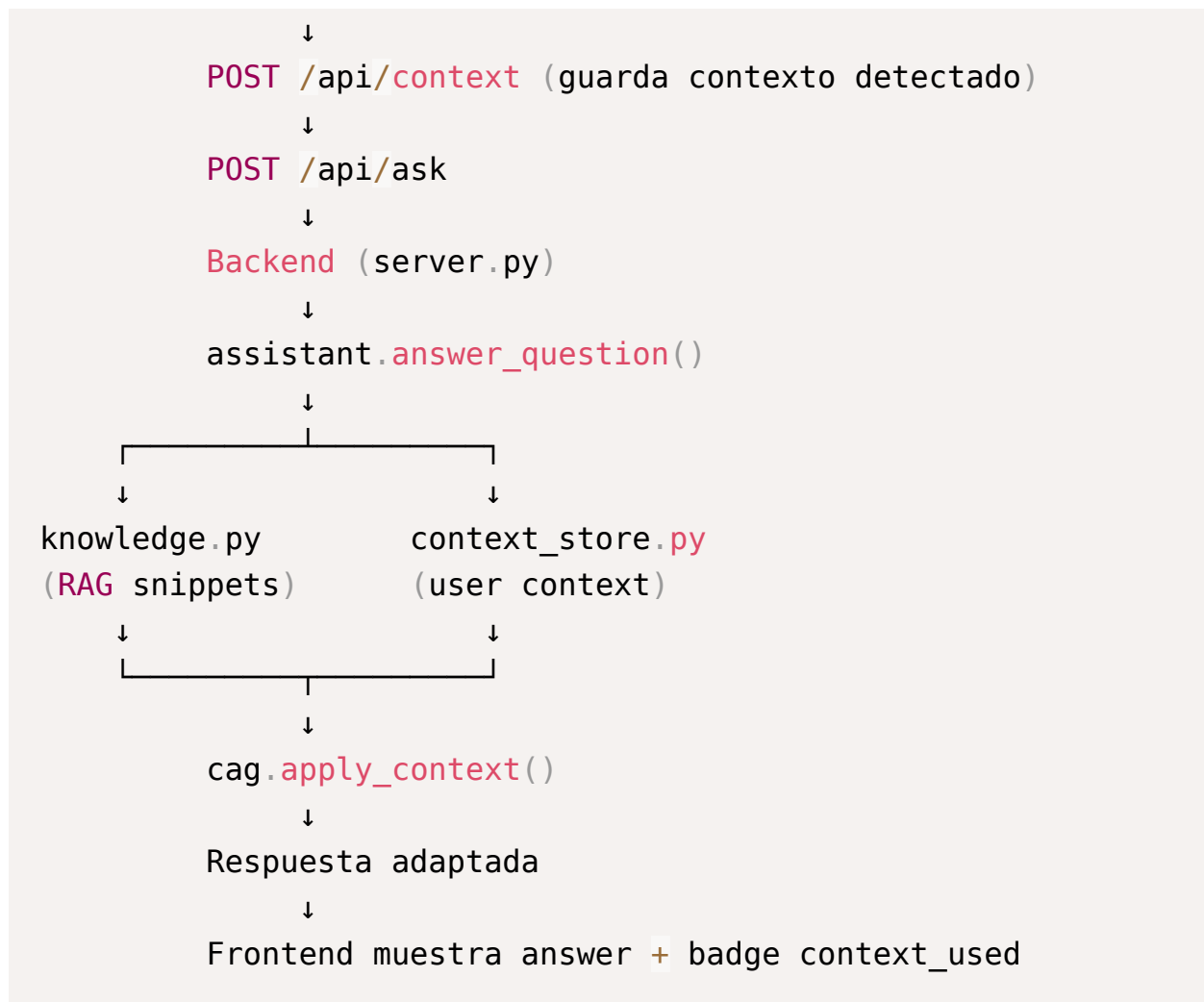
El resultado es un asistente que "recuerda" las preferencias del usuario entre mensajes, adaptando automáticamente las respuestas sin necesidad de repetir instrucciones.

---

## 2. Arquitectura del Sistema

### 2.1 Diagrama de flujo

```
Usuario → Frontend (app.js)
        ↓
        detectAndSaveContext()
```



## 2.2 Componentes principales

| Componente             | Archivo                               | Responsabilidad                      |
|------------------------|---------------------------------------|--------------------------------------|
| Servidor HTTP          | <code>backend/server.py</code>        | Endpoints REST, routing              |
| Asistente              | <code>backend/assistant.py</code>     | Orquestación RAG + CAG               |
| Base de conocimiento   | <code>backend/knowledge.py</code>     | Búsqueda por términos                |
| Almacén de contexto    | <code>backend/context_store.py</code> | Persistencia en memoria por usuario  |
| Aumentación contextual | <code>backend/cag.py</code>           | Reglas de modificación de respuestas |
| Frontend               | <code>frontend/app.js</code>          | Auto-detección, UI dinámica          |

### 3. Proceso de Desarrollo

El proyecto se desarrolló siguiendo **SDD (Spec-Driven Development)** y **TDD (Test-Driven Development)**.

#### Sprint 1: Backend — Módulo CAG

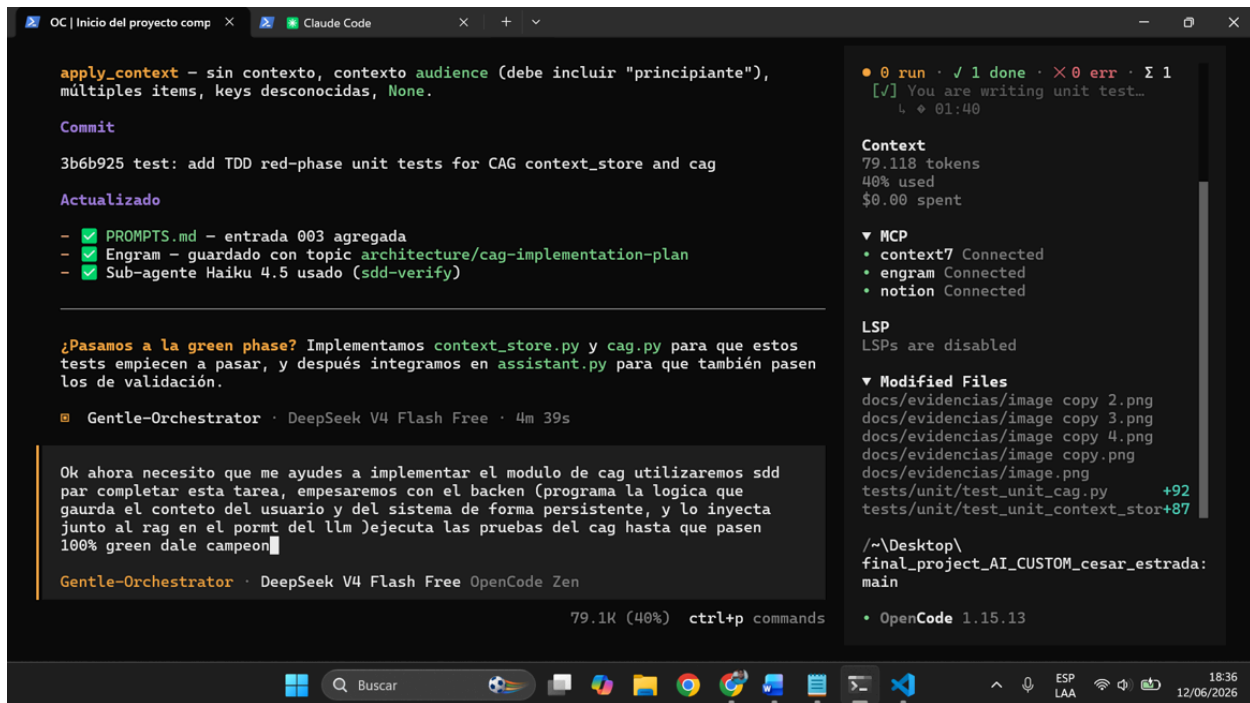
**Objetivo:** Implementar almacenamiento de contexto persistente y su inyección en respuestas.

##### Fases SDD ejecutadas:

1. **Init:** Detección de stack (Python stdlib, unittest)
2. **Explore:** Análisis de 10 archivos, identificación del pipeline
3. **Proposal:** Definición de alcance y no-goals
4. **Spec:** 9 requirements, 14 escenarios
5. **Design:** Dict storage, rule-based augmentation
6. **Tasks:** 21 tareas en 4 fases
7. **Apply:** Implementación TDD (red → green)
8. **Verify:** 17/17 tests green

##### Archivos creados/modificados:

- `backend/context_store.py` — ContextStore con dict `{user_id: {key: value}}`
- `backend/cag.py` — `apply_context()` con reglas
- `backend/assistant.py` — Integración singleton
- `tests/unit/test_unit_context_store.py` — 6 tests
- `tests/unit/test_unit_cag.py` — 5 tests



## Sprint 2: Frontend — Panel CAG y Auto-detección

**Objetivo:** Conectar frontend para visualizar retención de contexto.

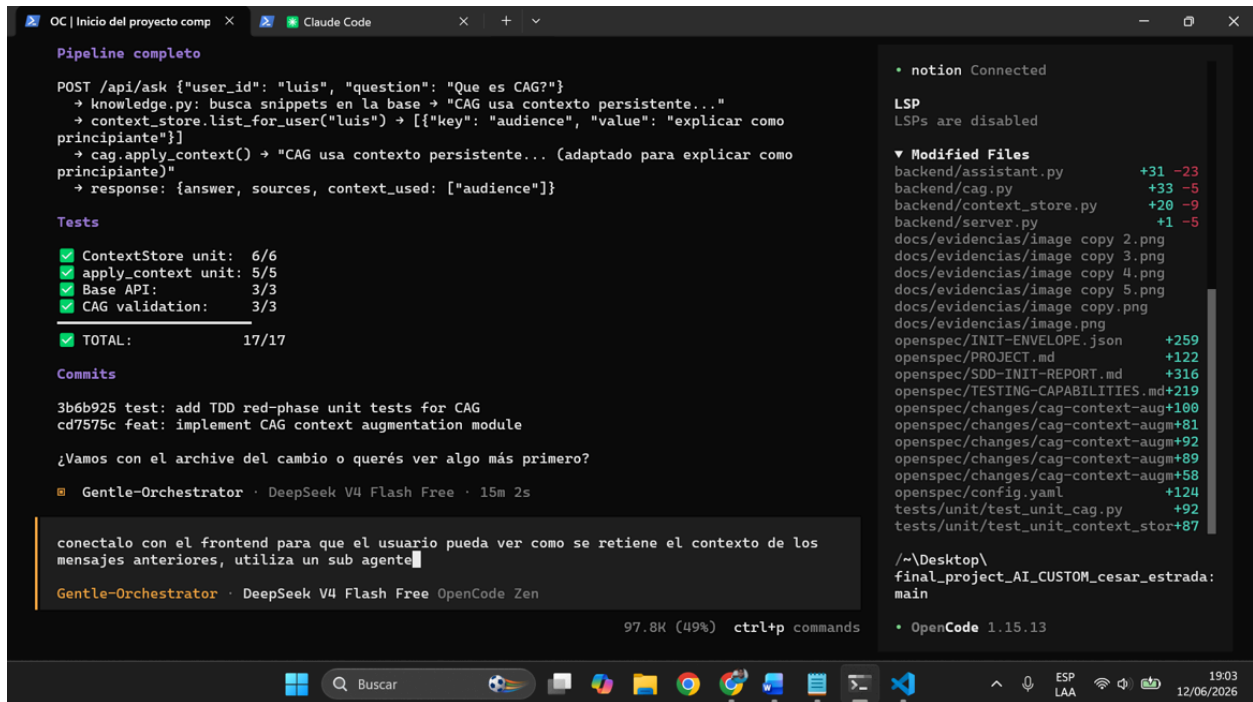
**Características implementadas:**

- Panel CAG con lista dinámica de contexto activo
- Formulario colapsable para agregar contexto manual
- **Auto-detección de contexto desde preguntas:**
  - "explicame como principiante" → `audience = principiante`
  - "en español" → `language = español`
  - "con ejemplos" → `include_examples = si`
- Badge que muestra `context_used` en cada respuesta
- Carga automática de contexto al abrir la página

**Archivos modificados:**

- `frontend/index.html` — Panel CAG rediseñado
- `frontend/app.js` — `detectAndSaveContext()`, `saveContext()`

- `frontend/styles.css` — Estilos para context-items, badges



## 4. API REST

| Método | Ruta                                  | Body                               | Respuesta                                       |
|--------|---------------------------------------|------------------------------------|-------------------------------------------------|
| GET    | <code>/health</code>                  | —                                  | <code>{"status": "ok"}</code>                   |
| POST   | <code>/api/ask</code>                 | <code>{user_id, question}</code>   | <code>{answer, sources, context_used}</code>    |
| POST   | <code>/api/context</code>             | <code>{user_id, key, value}</code> | <code>{"saved": true}</code>                    |
| GET    | <code>/api/context?user_id=...</code> | —                                  | <code>{user_id, context: [{key, value}]}</code> |

## 5. Contextos Soportados

| Clave                 | Valores                                    | Efecto en la respuesta                |
|-----------------------|--------------------------------------------|---------------------------------------|
| <code>audience</code> | <code>explicar como principiante</code>    | Agrega "(adaptado para principiante)" |
| <code>language</code> | <code>español</code> , <code>inglés</code> | Agrega "(en español)"                 |

| Clave                         | Valores         | Efecto en la respuesta            |
|-------------------------------|-----------------|-----------------------------------|
| <code>include_examples</code> | <code>si</code> | Agrega "(con ejemplos incluidos)" |

## 6. Tests

Estado: 17/17 tests green 

| Capa                 | Archivo                                 | Tests |
|----------------------|-----------------------------------------|-------|
| Unit — ContextStore  | <code>test_unit_context_store.py</code> | 6     |
| Unit — apply_context | <code>test_unit_cag.py</code>           | 5     |
| Base API             | <code>test_base_api.py</code>           | 3     |
| Validación CAG       | <code>test_cag_contract.py</code>       | 3     |

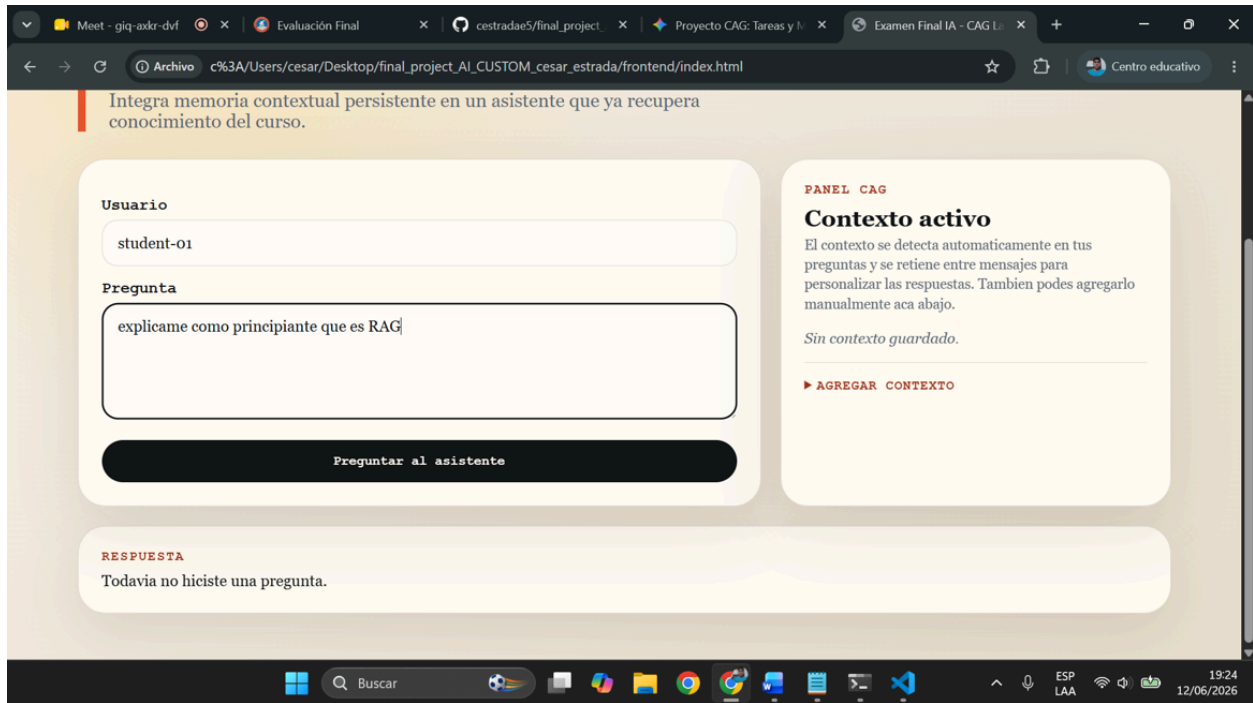
## 7. Ejemplo de Uso

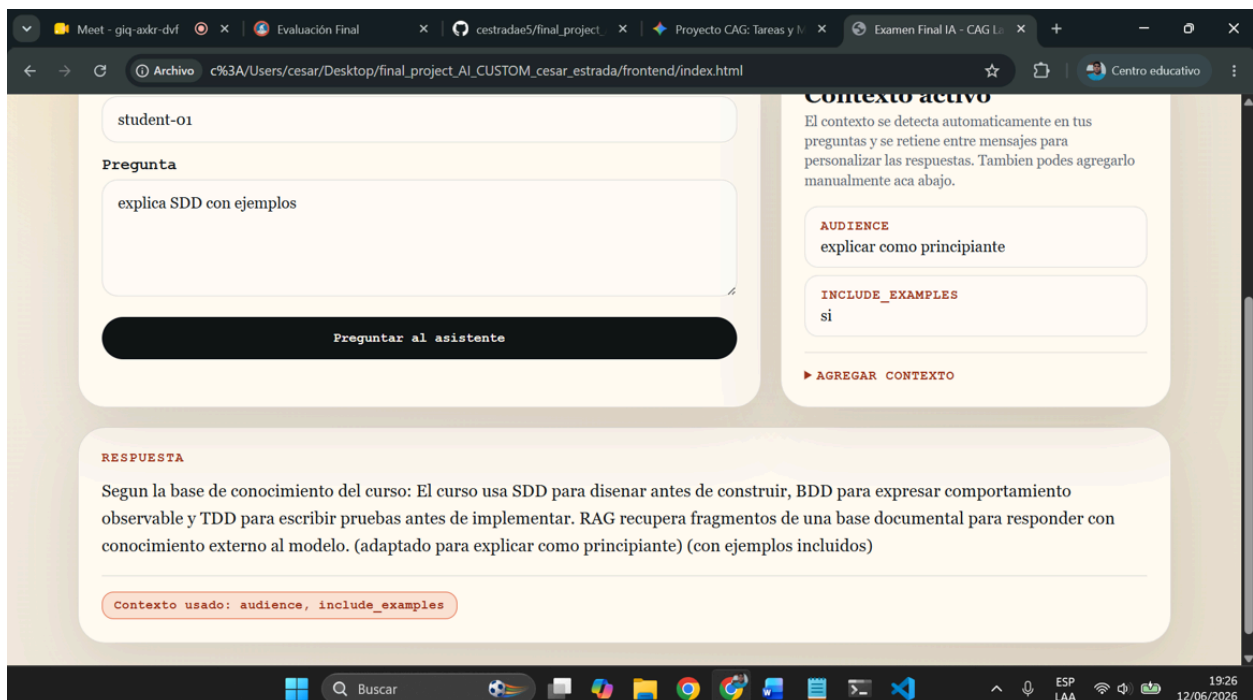
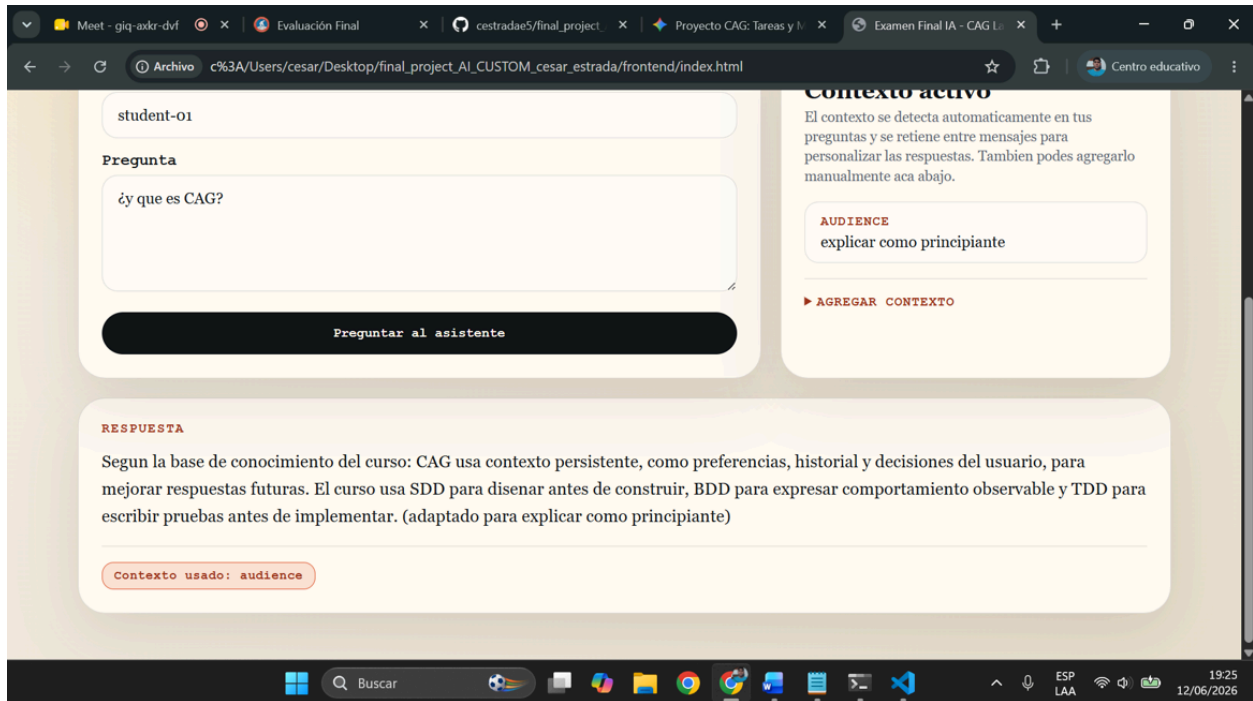
**PREGUNTA 1:** "explicame como principiante que es RAG"

- Auto-detect: audience = "explicar como principiante"
- Respuesta: "RAG recupera fragmentos... (adaptado para principiante)"
- Panel **CAG: AUDIENCE** | explicar como principiante
- Badge: [Contexto usado: audience]

**PREGUNTA 2:** "y que es CAG?"

- Sin nuevo contexto detectado
- **PERO** el contexto audience **PERSISTE**
- Respuesta: "CAG usa contexto... (adaptado para principiante)"
- Badge: [Contexto usado: audience] ← **RETENCIÓN**





## 8. Tecnologías y Herramientas

- **Lenguaje:** Python 3.14.6
- **Framework:** Ninguno (stdlib `http.server`)



- **Tests:** unittest (stdlib)
- **Frontend:** HTML5, CSS3, JavaScript vanilla
- **Metodología:** SDD (Spec-Driven Development) + TDD
- **IA asistente:** Claude (Opus, Sonnet 4.5, Haiku 4.5)

## 9. Estructura de Archivos

```

final_project_AI_CUSTOM_cesar_estrada/
├── backend/
│   ├── __init__.py
│   ├── server.py           # HTTP server + endpoints
│   ├── assistant.py        # Orquestación RAG + CAG
│   ├── knowledge.py         # Búsqueda en base de conocimiento
│   ├── context_store.py     # Almacén de contexto por usuario
│   └── cag.py               # Reglas de aumentación
├── frontend/
│   ├── index.html          # UI principal
│   ├── app.js              # Lógica + auto-detect
│   └── styles.css           # Estilos
├── data/
│   └── knowledge_base.json # Base de conocimiento (4 entradas)
├── tests/
│   ├── base/               # Tests base (3)
│   ├── unit/               # Tests unitarios TDD (11)
│   └── validation/         # Tests de contrato CAG (3)
├── openspec/               # Artefactos SDD
├── docs/                   # Documentación y evidencias
└── README.md               # Documentación principal

```

## 10. Conclusiones

El proyecto demuestra la implementación exitosa de un sistema CAG que:

1. **Retiene contexto** entre mensajes sin intervención manual del usuario
2. **Auto-detecta preferencias** desde el lenguaje natural de las preguntas
3. **Adapta respuestas** de forma visible y trazable (badge `context_used` )
4. **Sigue metodologías profesionales** (SDD, TDD, 17/17 tests green)
5. **Es completamente autónomo** (sin dependencias externas, puro stdlib)

El enfoque SDD garantizó que cada cambio estuviera especificado, diseñado y testeado antes de implementarse, resultando en un sistema robusto y mantenible.